

Wrocław University of Science and Technology

Faculty of Electronics, Photonics and Microsystems

Field of study: Automation and Robotics

Speciality: Embedded RObotics

Master Thesis

Title of Thesis:

**Analysis of Selected SLAM Methods
Using Various Sensor Types**

AUTHOR:

Marcin Tomczak

SUPERVISOR:

Janusz Jakubiak, PhD

WROCŁAW 2026

Abstract in Polish

Abstract in English

Contents

Abstract in Polish	1
Abstract in English	2
1 Introduction	5
1.1 Goal and Scope	6
1.2 Thesis Structure	7
2 Theoretical Background	8
2.1 Navigation in Mobile Robotics	8
2.1.1 Mapping	8
2.1.2 Localisation	9
2.1.3 SLAM	9
2.2 Sensors	9
2.2.1 LiDAR	9
2.2.2 Stereo Camera	10
2.2.3 IMU	10
2.2.4 Wheel Odometry	11
2.2.5 360-Degree Camera	11
2.3 Map Representations	11
2.3.1 Occupancy Grid	11
2.3.2 3D Point Cloud	12
2.4 Point Cloud Registration	12
2.4.1 K-d Tree	12
2.4.2 Iterative Closest Point (ICP)	13
2.4.3 Point-to-Plane ICP	13
2.4.4 Generalized ICP (GICP)	13
2.4.5 Normal Distributions Transform (NDT)	13
2.5 Filter-Based State Estimation	14
2.5.1 Extended Kalman Filter (EKF)	14

CONTENTS	4
2.5.2 Rao-Blackwellized Particle Filter	14
2.6 Graph-Based SLAM	14
2.6.1 Pose Graphs	15
2.6.2 Keyframes	15
2.7 Loop Closure Detection	15
2.7.1 Branch-and-Bound Search	16
2.7.2 Bag-of-Words Visual Place Recognition	16
2.8 Visual SLAM Fundamentals	16
2.8.1 Stereo Vision and Triangulation	16
2.8.2 Feature Detection and Description	17
2.8.3 Visual Odometry	17
2.9 Evaluation Metrics	17
2.9.1 Umeyama Alignment	18
2.9.2 Absolute Trajectory Error (ATE)	18
2.9.3 Relative Pose Error (RPE)	18
2.9.4 Chamfer Distance	19
2.9.5 F-score	19
3 Algorithm description	20
3.1 Sensor Fusion	20
3.1.1 EKF Dead Reckoning	21
3.2 LiDAR Odometry	21
3.2.1 KISS-ICP	21
3.2.2 MOLA Odometry	21
3.3 2D LiDAR SLAM	21
3.3.1 Google Cartographer 2D	22
3.3.2 SLAM Toolbox	22
3.3.3 GMapping	23
3.4 3D LiDAR SLAM	23
3.4.1 RTABMap (LiDAR)	23
3.5 Tightly-Coupled LiDAR-IMU	23
3.5.1 FAST-LIO2	24
3.6 Visual SLAM	24
3.6.1 cuVSLAM	24
3.6.2 RTABMap RGB-D	25
Bibliography	26
A Appendices	30

Chapter 1

Introduction

Simultaneous Localisation and Mapping (SLAM) is one of the fundamental problems in mobile robotics [1]. SLAM poses a question: how a robot can build a consistent map of an unknown environment while simultaneously estimating its own position within that map. *Localisation* refers to estimating the state of the robot, which consists of position and orientation of the robot. *Mapping* refers to construction of a spatial model of the surrounding environment. Having a map is useful for two reasons [2]. First, it enables downstream tasks such as navigation planning and environment visualisation for a human user. Second, it keeps localisation errors in check by letting the robot recognise where it has already been. SLAM is therefore considered a state of the art for autonomous robots that must operate without GPS.

The difficulty of the SLAM problem lies in the mutual dependency of its two components. A good map can only be built if the robot knows its position, and the robot can only determine its position if it already has a map. Both must therefore be estimated at the same time from noisy sensor data. The key mechanism that prevents errors from growing indefinitely is *loop closure*: when the robot returns to a place it has already visited, it detects the match and uses it to correct the drift that has built up in its trajectory. Without it, errors accumulate without bound [2].

The origins of SLAM research trace back to 1986, when Smith, Self, and Cheeseman [3], and independently Durrant-Whyte [4], first described geometric uncertainty and the statistical relationships between robot poses and environmental landmarks. The term *SLAM* itself was coined at the International Symposium on Robotics Research (ISRR) in 1995 [5]. Early work introduced the main probabilistic formulations: Extended Kalman Filters, Rao-Blackwellised Particle Filters, and maximum

likelihood estimation. In following years, research focused on observability, convergence, and consistency, which produced the first widely used open-source libraries [2]. Since then, supported sensors have expanded from 2D laser scanners to 3D LiDARs, cameras, and tightly-coupled inertial systems, and the current research focus has shifted towards robust long-term operation.

SLAM has been evolving since its formulation four decades ago, driven largely by the growing use of mobile robots in indoor environments such as homes or warehouses, where GPS is not available. Today it is used across many domains: robotic vacuum cleaners navigate home environments, autonomous vehicles map urban roads, unmanned aerial vehicles use it for inspection and surveillance, and underwater drones can monitor coral reefs or detect mines.

Despite four decades of progress, SLAM is not a fully solved problem [2]. It can work reliably for slow-moving robots in static indoor environments, but can struggle in fast-moving or highly dynamic outdoor scenarios. Many open-source implementations exist, each making different design choices with different trade-offs. Choosing the right algorithm for a given robot and environment is not obvious, which motivates a direct, controlled comparison.

1.1 Goal and Scope

The goal of this thesis is to evaluate and compare SLAM algorithms across different sensor modalities and algorithmic families under identical, controlled conditions. The evaluation is carried out on a simulated mobile robot platform equipped with differential drive, a 3D LiDAR, a stereo camera, and an IMU. Ten algorithms are selected to cover the major algorithmic families: sensor fusion, LiDAR odometry, 2D and 3D LiDAR SLAM, tightly-coupled LiDAR-IMU odometry, and visual SLAM.

All algorithms are tested on the same dataset recorded from simulated environment in NVIDIA Isaac Sim, which provides exact ground-truth poses for objective evaluation. The framework used for all experiments is ROS 2 (Robot Operating System), which provides a common interface for sensor data, algorithm execution, and data recording.

Trajectory accuracy is quantified using Absolute Trajectory Error and Relative Pose Error against the simulator’s ground truth. Map quality is assessed with Chamfer distance and F-score metrics for algorithms that produce a 3D point cloud, and visually for those that produce an occupancy grid. Computational cost is measured as mean CPU usage and peak RAM consumption during execution.

1.2 Thesis Structure

Chapter 2 contains the theoretical background required to understand the thesis: sensors, map representations, point cloud registration, state estimation, graph-based SLAM, loop closure detection, and evaluation metrics.

Chapter 3 describes each of the ten evaluated algorithms, grouped by method family.

Chapter 4 presents the experimental setup, the dataset, and the results for all ten algorithms, including trajectory accuracy, map quality, and computational cost.

Chapter 5 concludes the thesis and provides a summary of findings made during experimental evaluation of algorithms.

Chapter 2

Theoretical Background

2.1 Navigation in Mobile Robotics

Navigation is "the ability of the robot to act based on its knowledge and sensor values so as to reach its goal positions as efficiently and as reliably as possible" [6, p. 257]. It can be decomposed into five components:

1. Perception: recording and interpreting raw sensor data such as point clouds or images.
2. Localisation: estimating the robot pose within a known environment.
3. Mapping: constructing a spatial model of the environment from sensor observations.
4. Path planning: computing a collision-free route from the current position to a goal.
5. Motion control: generating actuator commands that follow the planned path while reacting to unexpected obstacles.

SLAM combines components 2 and 3, estimating the robot pose and the map simultaneously without prior knowledge.

2.1.1 Mapping

Mapping is the problem of building a spatial model of the environment from sensor measurements, assuming the robot trajectory is known [7, p. 221]. Even with exact poses the problem is non-trivial. Sensor noise and visually similar places can cause incorrect data association, and any pose error introduces geometrical

distortion across the map. Common map representations used in this thesis are the 2D occupancy grid and the 3D point cloud, described in Section 2.3.

2.1.2 Localisation

Localisation is the complementary problem to mapping. Given a known map, the robot determines its current pose by fusing sensor measurements with motion commands [7, p. 157]. Because pose cannot be read from a sensor directly, the estimate is built incrementally as each new measurement updates the belief. All algorithms evaluated in this thesis operate with a known starting position (position tracking mode) [7, p. 159]: the search is confined to a small neighbourhood of the current estimate, which keeps computational cost low.

2.1.3 SLAM

SLAM assumes that neither the robot poses $\mathbf{x}_{1:t}$ nor the map $\mathbf{m}$ are known, and estimates both jointly from sensor measurements $\mathbf{z}_{1:t}$ and motion commands $\mathbf{u}_{1:t}$ [7, p. 245]. Two forms of the problem are common [7, pp. 245 and 246]. Online SLAM discards past poses and maintains only the current pose together with the map:

$$p(\mathbf{x}_t, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}). \quad (2.1)$$

Full SLAM keeps all past poses and estimates the complete trajectory together with the map:

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}). \quad (2.2)$$

The online form is recovered from the full one by integrating out past poses. The two dominant solution families of SLAM problem, filter-based and graph-based methods, are described in Sections 2.5 and 2.6 respectively.

2.2 Sensors

Sensors provide the data necessary for SLAM implementation. This thesis uses four sensor types: a 3D LiDAR, a stereo camera, an IMU, and wheel odometry encoders, each described in its subsection below.

2.2.1 LiDAR

A LiDAR (Light Detection And Ranging) sensor measures distances by emitting laser pulses and recording the round-trip travel time Δt . The distance to the reflecting surface is given by:

$$d = \frac{c \cdot \Delta t}{2}, \quad (2.3)$$

where c is the speed of light. A rotating 3D LiDAR fires pulses across multiple vertical channels while spinning horizontally. Each full revolution produces a cloud of tens of thousands of (x, y, z) points. Its main advantages for SLAM are accuracy and independence from ambient lighting. The main drawback is continuous movement of the robot during a single scan. As a result, points from the same scan are captured from slightly different poses, making the resulting cloud geometrically inconsistent without correction.

2.2.2 Stereo Camera

A stereo camera consists of two horizontally separated cameras with a known baseline B between them. When the same scene point is visible in both images it appears at slightly different horizontal positions. This horizontal offset d allows the depth Z to be computed as

$$Z = \frac{f \cdot B}{d}, \quad (2.4)$$

where f is the focal length in pixels.

Cameras capture dense appearance information from which features can be extracted for feature matching and place recognition. However, if the environment lacks distinctive texture, stereo vision fails to find reliable matches [6, p. 113].

2.2.3 IMU

An Inertial Measurement Unit (IMU) combines a three-axis accelerometer and a three-axis gyroscope. Both outputs are corrupted by a slowly varying bias $\mathbf{b}$ and noise $\boldsymbol{\eta}$:

$$\tilde{\mathbf{a}} = \mathbf{a} + \mathbf{b}_a + \boldsymbol{\eta}_a, \quad \tilde{\boldsymbol{\omega}} = \boldsymbol{\omega} + \mathbf{b}_g + \boldsymbol{\eta}_g, \quad (2.5)$$

where $\tilde{\mathbf{a}}$ and $\tilde{\boldsymbol{\omega}}$ are the measured acceleration and angular velocity, and $\mathbf{a}$, $\boldsymbol{\omega}$ are the true values. The IMU runs at a higher rate than other sensors, providing motion information between slower LiDAR or camera updates. However, integrating corrupted signals accumulates errors over time, so the IMU must be combined with another sensor to remain accurate [8].

2.2.4 Wheel Odometry

Wheel encoders measure the angular velocity of each wheel, from which the linear speeds v_L and v_R of the left and right wheels are calculated. For a differential-drive robot moving at heading θ , the pose increment over a time step Δt is given by:

$$\Delta x = \frac{v_L + v_R}{2} \Delta t \cos \theta, \quad \Delta y = \frac{v_L + v_R}{2} \Delta t \sin \theta, \quad \Delta \theta = \frac{v_R - v_L}{L} \Delta t, \quad (2.6)$$

where L is the distance between the wheels. Integrating these increments over time provides a pose estimate. However, wheel slip and calibration errors in the wheel radius and distance between them causes the estimate to drift over time [6, Ch. 3].

2.2.5 360-Degree Camera

A 360-degree (omnidirectional) camera captures the full horizontal field of view in a single frame, either through a fisheye lens or by stitching multiple camera feeds into one projection. The full environment coverage reduces the risk of losing tracked features during sharp rotations. The main drawbacks are strong lens distortion and the absence of a single agreed sensor model, since the term covers fisheye, catadioptric (camera with mirror), and multi-camera designs without a unified definition. This sensor type is not evaluated in this thesis: stereo camera-based visual SLAM is used instead, and Isaac Sim does not provide a native omnidirectional camera [6, Ch. 4].

2.3 Map Representations

The map representation affects memory usage, update speed, and available downstream operations. The algorithms evaluated in this thesis produce either a 2D occupancy grid or a 3D point cloud.

2.3.1 Occupancy Grid

An occupancy grid divides the 2D plane into a regular grid of square cells, each storing the probability $p(m_i \mid \mathbf{z}_{1:t}, \mathbf{x}_{1:t})$ that cell m_i is occupied, given all measurements and poses up to time t [7]. Rather than storing the probability directly, cells are updated using the log-odds representation

$$l_{t,i} = \log \frac{p(m_i \mid \mathbf{z}_{1:t}, \mathbf{x}_{1:t})}{1 - p(m_i \mid \mathbf{z}_{1:t}, \mathbf{x}_{1:t})}, \quad (2.7)$$

Cells with log-odds above a threshold are marked occupied (obstacles), those below a second threshold are marked free, and the rest remain unknown. Occupancy grids directly support path planning, as each cell indicates whether the region is available for the robot. The representation is limited to 2 dimensions and the cell size, fixed as a parameter before mapping, determines the spatial resolution of the map.

2.3.2 3D Point Cloud

A 3D point cloud is an unordered set of points in a common reference frame, each defined by its (x, y, z) coordinates [2]. A global map is built by registering each incoming LiDAR scan into the map frame using the estimated robot pose and accumulating the resulting points. The resulting map captures the full three-dimensional geometry of the environment.

Point clouds are well suited for geometric inspection and for evaluating map quality against a reference scan. Their drawbacks are high memory consumption in large environments and the absence of information about free-space: unlike an occupancy grid, a point cloud records only observed surfaces, not which regions are reachable for the robot.

Visual SLAM methods do not accumulate dense range measurements. Instead, they build sparse point clouds by triangulating matched image features into 3D landmarks. The density of such a map depends on the visual texture of the environment, therefore it is unsuitable for measurable geometric evaluation and is not evaluated as such in this thesis.

2.4 Point Cloud Registration

Point cloud registration finds the rigid transformation that best aligns two overlapping point clouds. In LiDAR SLAM each new scan is aligned to the previous one or to the accumulated map to estimate motion.

2.4.1 K-d Tree

Many registration and evaluation methods need to find the closest point in a target cloud for each point in a source cloud. Comparing against every stored point is too slow when clouds contain tens of thousands of points.

A k-d tree [9] solves this by organising points into a hierarchical structure that allows most of the search space to be skipped, reducing the cost from $O(n)$ to $O(\log n)$ per query. ICP and Chamfer Distance both rely on this structure.

2.4.2 Iterative Closest Point (ICP)

ICP [10] is a general method for registering 3D shapes. The algorithm alternates between two steps. First, each point in the source cloud is matched to its nearest neighbour in the target cloud. Second, the rigid transformation that minimises the sum of squared distances between matched pairs is computed. These steps repeat until the change in error falls below a predefined threshold. The solution can get trapped in a local minimum rather than finding the globally correct alignment. A poor initial estimate leads to wrong point pairs being formed, causing the algorithm to converge to an incorrect result.

2.4.3 Point-to-Plane ICP

Point-to-plane ICP [11] extends standard ICP by replacing the point-to-point distance with the distance to the local surface at each matched target point. The surface is estimated from the neighbouring points of the target using a surface normal. This means only motion away from the surface is penalised, while motion along it is unconstrained. In environments with many walls, floors, and ceilings this leads to faster convergence than standard ICP. MOLA Odometry uses point-to-plane ICP.

2.4.4 Generalized ICP (GICP)

GICP [12] extends point-to-plane ICP by treating both the source and the target as locally flat surfaces rather than just the target. Each point carries a small description of the surface patch around it, and matching minimises the misalignment between these patches on both sides. This makes the algorithm less sensitive to noisy matches and to the threshold used for accepting a correspondence. GICP is the default registration method of KISS-ICP.

2.4.5 Normal Distributions Transform (NDT)

NDT [13] avoids finding point pairs entirely. The target cloud is divided into a 3D voxel grid. For each voxel, the average position and the spread of all points inside it are computed, describing where the surface is and its local shape. To find the alignment, each source point is transformed and the algorithm checks how well it fits into the surface description of the voxel it lands in. The transformation is adjusted until the source points fit the target descriptions as well as possible. Because the fit measure changes smoothly as the transformation changes, the optimisation is more stable than ICP, which can jump when matched pairs suddenly switch. MOLA Odometry uses NDT.

2.5 Filter-Based State Estimation

Filter-based methods maintain a probability distribution over the robot state and update it recursively as new sensor data arrives. The update cycle alternates between a prediction step, which propagates the state forward using a motion model, and a correction step, which incorporates the latest sensor measurement to reduce uncertainty.

2.5.1 Extended Kalman Filter (EKF)

The standard Kalman filter assumes that both the motion model and the sensor model are linear, which is rarely true in robotics. The EKF [7, pp. 48 to 65] solves this by approximating the nonlinear models locally at each step, making it applicable to real robot motion and sensor fusion. It maintains a best estimate of the robot state together with a measure of confidence in that estimate. The filter alternates between two steps. In the prediction step, the motion model is used to estimate where the robot moved to, causing the confidence to decrease. In the correction step, a new sensor measurement is used to pull the estimate toward what the sensor observed, and the confidence increases.

The iterated variant (iEKF) [14] improves on this by repeating the correction step several times, each time re-approximating the sensor model at the latest corrected estimate rather than the original one. This gives a more accurate result when the sensor model is strongly nonlinear.

2.5.2 Rao-Blackwellized Particle Filter

The robot state in a Rao-Blackwellized Particle Filter [15] is represented as a set of particles, each containing a possible trajectory and a separate map built from that trajectory. When the robot moves, all particles are updated according to the motion model, each in a slightly different way to reflect motion uncertainty. When a new sensor scan arrives, each particle checks how well the scan fits its own map and receives a score. Particles with a poor score are discarded and replaced by copies of high-scoring ones, so the set concentrates on the most plausible trajectories. The final map is taken from the best-scoring particle. GMapping, evaluated in this thesis, applies this approach to occupancy-grid SLAM [16].

2.6 Graph-Based SLAM

Most SLAM systems are structured as a two-stage pipeline. The front end processes raw sensor data to extract relative pose constraints (through scan matching, visual

feature tracking, or IMU integration) and identifies loop closure candidates. The back end takes these constraints as input and optimises the full trajectory and map for global consistency, typically by solving a nonlinear least-squares problem over a pose or factor graph. In contrast to filter-based methods, which process measurements sequentially and discard past information, graph-based approaches retain the full trajectory and can enforce global consistency whenever a loop closure is detected [17].

2.6.1 Pose Graphs

A pose graph [17] represents the robot trajectory as a set of nodes, one for each robot pose, connected by edges. Each edge records the relative movement between two poses as measured by odometry or scan matching, together with a confidence value. When a loop closure is detected, a new edge is added connecting the current pose to the earlier matching pose. The graph is then optimised to find the set of poses that best satisfies all constraints simultaneously, correcting the drift accumulated along the trajectory. Cartographer 2D, SLAM Toolbox, and RTABMap all use pose graph back-ends.

2.6.2 Keyframes

A keyframe is a selected frame from the robot’s trajectory that is stored as a reference point for matching and optimisation. Not every frame becomes a keyframe. A new keyframe is typically created only when the robot has moved or rotated enough since the last one. Storing only keyframes keeps the pose graph small and the optimisation fast, without losing the information needed to correct drift. In the pose graph, each node usually corresponds to a keyframe. IMU measurements between two keyframes are accumulated into a single summary, and visual odometry uses a small group of recent keyframes to refine the current pose estimate.

2.7 Loop Closure Detection

Loop closure detection [2] identifies when the robot has returned to a previously visited location. Small errors accumulate with each step, so the estimated position can drift significantly over a long trajectory. When a revisit is detected, a new edge is added to the pose graph connecting the current pose to the earlier one, and the graph is re-optimised to distribute the correction across the full trajectory. How candidates are identified depends on the algorithm. SLAM Toolbox searches nearby nodes in the pose graph. Cartographer searches all previously built submaps using branch-and-bound, described below. RTABMap compares a compact descriptor

of the current scan against all stored nodes, so it can detect a revisit even after a long detour. For visual systems, bag-of-words place recognition is used, described in Section 2.7.2.

2.7.1 Branch-and-Bound Search

Cartographer uses branch-and-bound [18] to find the best alignment between an incoming scan and a stored submap. The pose space is divided into blocks. For each block, the algorithm estimates the best score that any pose inside it could achieve. If that estimate is already worse than the current best, the entire block is discarded without further search. This guarantees a globally optimal result while avoiding an exhaustive search, unlike ICP which depends on the starting position and can miss the best answer.

2.7.2 Bag-of-Words Visual Place Recognition

Bag-of-words (BoW) place recognition [19] represents each image as a compact list of visual patterns and how often they appear. These patterns, called visual words, are learned in advance from a large set of images. When the robot sees a new place, its image is converted into this representation and compared against all stored images in the map. This comparison is fast because it involves only counting matching visual words rather than comparing raw pixel values. A stored image that scores high enough becomes a loop closure candidate. Before accepting it, the system verifies that the two images can be geometrically aligned. Only if that check passes is the loop closure edge added to the pose graph. RTABMap uses this approach for loop closure detection.

2.8 Visual SLAM Fundamentals

Visual SLAM estimates the robot trajectory from image sequences by detecting and tracking sparse visual features rather than measuring range directly. This thesis evaluates feature-based stereo systems, which recover depth from stereo disparity (Section 2.2) and estimate pose from the 3D positions of matched keypoints.

2.8.1 Stereo Vision and Triangulation

Stereo depth is computed from the disparity between the two images as given by Equation (2.4) in Section 2.2. Each matched feature pair is converted to a 3D landmark by tracing both camera rays and finding their intersection [20, Ch. 11].

Depth uncertainty grows quadratically with distance, so visual SLAM front-ends typically discard landmarks beyond a sensor-specific threshold.

2.8.2 Feature Detection and Description

Visual feature detection finds distinctive points in an image and describes each one in a way that allows it to be recognised in later frames.

A good feature to track is a corner, not an edge. On an edge, a feature can slide along it without the tracker noticing. A corner is uniquely located and stays put. The Good Features to Track (GFTT) detector [21] selects corners, points that look distinct from every direction and therefore cannot be confused with neighbouring points. GFTT is commonly used in visual SLAM systems where reliable tracking is more important than distinctive appearance description.

2.8.3 Visual Odometry

Visual odometry estimates the robot’s motion by tracking features across consecutive camera frames [22]. Matched features are converted to 3D landmarks and used to compute the pose change between frames. A small group of recent keyframes is kept and jointly refined to reduce accumulated error. Without loop closure, small errors add up with every step, so drift grows steadily over a long trajectory.

2.9 Evaluation Metrics

Two groups of metrics are used in this thesis. Trajectory metrics measure how closely the estimated robot path matches the ground truth poses recorded from the simulation. Map metrics measure the geometric accuracy of the 3D point cloud produced by each algorithm relative to the reference map from the simulator. The ground truth is a time-stamped sequence of $SE(3)$ poses, one per sensor frame, taken directly from the simulator with no sensor noise.

Each metric produces a per-pose or per-point error value. These are summarised by three statistics. The RMSE (Root Mean Square Error) is the square root of the average squared error and is the primary reported value, as it penalises large errors more than small ones. The mean is the simple average of all error values. The maximum is the single largest error in the sequence and indicates worst-case behaviour.

2.9.1 Umeyama Alignment

A SLAM algorithm starts its own coordinate frame from the first sensor measurement, so the estimated trajectory may be shifted or rotated relative to the ground truth. Before measuring errors, the two trajectories are aligned using the method of Umeyama [23], which finds the rotation, translation, and scale factor that bring them as close together as possible. All sensors used in this thesis produce metric measurements, so the scale is fixed to one and only the orientation and position offset are corrected.

2.9.2 Absolute Trajectory Error (ATE)

ATE measures how closely the estimated trajectory matches the ground truth over the full run [24]. After Umeyama alignment, the error at each pose is the straight-line distance between the estimated position and the ground-truth position. The RMSE over all n poses is

$$\text{ATE} = \sqrt{\frac{1}{n} \sum_{i=1}^n \|\hat{\mathbf{p}}_i - \mathbf{p}_i^*\|^2}, \quad (2.8)$$

where $\hat{\mathbf{p}}_i$ is the estimated position and $\mathbf{p}_i^*$ is the ground-truth position at time step i . A large ATE indicates that the algorithm drifts over the full trajectory, or that loop closure is absent or ineffective.

2.9.3 Relative Pose Error (RPE)

RPE measures local accuracy step by step, without depending on the global alignment [24]. For each consecutive pose pair the motion predicted by the estimate is compared to the actual motion in the ground truth. The translational difference between the two is the local error for that step. The RMSE over all $m = n - 1$ steps is

$$\text{RPE} = \sqrt{\frac{1}{m} \sum_{i=1}^m \|\mathbf{e}_i^{\text{trans}}\|^2}, \quad (2.9)$$

where $\mathbf{e}_i^{\text{trans}}$ is the translational difference between the estimated and ground-truth one-step motions. Because RPE is computed step by step, it captures short-term drift that ATE may hide when loop closure corrects the global trajectory.

2.9.4 Chamfer Distance

The Chamfer Distance (CD) measures how well two point clouds match each other [25]. For each point in the estimated cloud the distance to the nearest point in the reference cloud is found, and vice versa. CD sums the mean distances in both directions:

$$d_{CD}(S_1, S_2) = \frac{1}{|S_1|} \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \frac{1}{|S_2|} \sum_{y \in S_2} \min_{x \in S_1} \|y - x\|_2. \quad (2.10)$$

where S_1 is the estimated cloud, S_2 is the reference cloud, and x, y denote individual points in each cloud.

The first term catches extra points in the estimate that are far from the reference. The second term catches parts of the reference that the estimate misses. CD is reported in metres, but a small number of outlier points can inflate the score without reflecting a real mapping failure.

2.9.5 F-score

The F-score at a distance threshold d combines precision and recall into a single quality measure [26]. Precision measures the accuracy of the reconstruction: how many estimated points are close to the ground truth. Recall measures completeness: how much of the ground truth is covered by the estimate. Both are computed as the fraction of points within distance d of the other cloud. They are combined into a single score:

$$F(d) = \frac{2 P(d) R(d)}{P(d) + R(d)}. \quad (2.11)$$

Three thresholds are used in this thesis: 5 cm, 10 cm, and 20 cm. A tight threshold such as 5 cm only counts a point as correctly placed if it is very close to the reference. A loose threshold such as 20 cm is more forgiving and rewards overall completeness. The F-score is in $[0, 1]$, where 1 means a perfect match.

Chapter 3

Algorithm description

Here add why these algorithms were chosen.

This chapter describes each of the 10 algorithms evaluated in this thesis, grouped by method family. Table 3.1 lists them with their sensors and method families.

Table 3.1: Evaluated algorithms overview

#	Algorithm	Sensors	Method family
1	EKF Dead Reckoning	Wheel + IMU	Sensor fusion
2	KISS-ICP	LiDAR	LiDAR odometry
3	MOLA Odometry	LiDAR	LiDAR odometry
4	Cartographer 2D	LiDAR (2D scan)	2D LiDAR SLAM
5	GMapping	LiDAR (2D scan)	2D LiDAR SLAM
6	SLAM Toolbox	LiDAR (2D scan)	2D LiDAR SLAM
7	RTABMap (LiDAR)	LiDAR	3D LiDAR SLAM
8	FAST-LIO2	LiDAR + IMU	LiDAR-IMU odometry
9	cuVSLAM	Stereo camera	Visual SLAM
10	RTABMap RGB-D	Stereo camera	Visual SLAM

3.1 Sensor Fusion

Sensor fusion combines wheel odometry and IMU to estimate pose without building a map. It serves as a dead-reckoning baseline.

3.1.1 EKF Dead Reckoning

EKF (Extended Kalman Filter) node from the `robot_localization` ROS2 package fuses wheel odometry and IMU data to estimate the robot's pose without any geometric map. Dead reckoning accumulates drift from wheel slip and noisy IMU data, so errors grow unboundedly over time unless corrected.

3.2 LiDAR Odometry

LiDAR odometry methods estimate the motion of the robot by aligning consecutive LiDAR scans using the Iterative Closest Point (ICP) algorithm. Each new scan is compared to a local map of the previous few scans. The best-fitting transformation is taken as the pose change.

These methods do not build a global map and do not detect loop closures, which leads estimation error to accumulate proportionally to travelled distance.

LiDAR odometry is computationally efficient and works without visual features. Methods that add loop closure or IMU fusion should outperform it, especially on longer trajectories.

3.2.1 KISS-ICP

KISS-ICP [27] (Keep It Small and Simple ICP) uses GICP with adaptive parameters, requiring no manual tuning. It was published in 2022 by researchers at the University of Bonn.

3.2.2 MOLA Odometry

MOLA [28] (Modular Optimization framework for Localization and mApping) was developed at the University of Almería. Only its LiDAR odometry component (`mola_lidar_odometry`) was used here.

It uses point-to-plane ICP with an NDT voxel map. Older scans are discarded as the robot moves to limit memory usage. MOLA is built as a modular pipeline, which makes components easy to swap but requires more configuration than KISS-ICP.

3.3 2D LiDAR SLAM

2D LiDAR SLAM is designed for planar laser scanners that produce a single horizontal sweep of range measurements. Representing the environment as a horizontal plane reduces the pose estimation problem from 6-DoF to 3-DoF (x , y ,

θ). This simplification lowers computational cost and is valid for wheeled robots on flat floors, but it discards all height information. 2D SLAM is therefore unsuitable when full spatial reconstruction is needed or when the robot operates on uneven terrain. The TurtleBot3 platform evaluated in this thesis is a wheeled robot operating on a flat floor, so the planar LiDAR is sufficient.

The simulation used in this thesis does not have a 2D laser scanner, only a point cloud from a 3D LiDAR. The lack of 2D LiDAR in simulation is caused by computational limitations. Data from 3D LiDAR was processed offline: one horizontal ring was extracted from each 3D scan and saved into a separate pre-processed bag. All three 2D SLAM algorithms read from this bag.

Cartographer and SLAM Toolbox both build pose graphs and run global optimisation when a loop closure is detected. GMapping uses a particle filter, where each particle carries its own map.

3.3.1 Google Cartographer 2D

Cartographer 2D [18] was developed by Google and uses a two-stage architecture. The local stage matches each incoming scan against a small occupancy submap. Once a submap is complete it is frozen and becomes a node in the pose graph. The global stage searches for loop closures between frozen submaps using a branch-and-bound matcher, which is globally optimal. Accepted loop closures are added as edges and the pose graph is then optimised. The final map is a single occupancy grid assembled from all submaps.

3.3.2 SLAM Toolbox

SLAM Toolbox [29] was developed by Steve Macenski and is the recommended 2D SLAM solution for ROS 2 Navigation2. Its defining feature is lifelong mapping: the pose graph and map can be serialised to disk and resumed in a later session, which makes it well suited to large environments that cannot be mapped in a single run.

Scan matching uses a correlation-based matcher: a grid of candidate poses is evaluated and the one that maximises the overlap score between the incoming scan and the stored map is selected as the best alignment. Unlike Cartographer's branch-and-bound approach, this can get stuck in local minima, making loop closure more sensitive to tuning and environment geometry.

3.3.3 GMapping

GMapping [16] is a Rao-Blackwellized particle filter SLAM. There is no explicit loop closure. When the robot revisits a known area, particle weights naturally shift toward poses consistent with the existing map, which corrects the trajectory. This makes GMapping more tolerant of repetitive environments where a correlation-based matcher might commit to a false loop, but it also limits correction ability on long trajectories.

GMapping officially targets ROS 1. The ROS 2 port used here was compiled from source against the `openslam_gmapping` library.

3.4 3D LiDAR SLAM

3D LiDAR SLAM builds maps from the full point cloud without discarding spatial information, making it suitable when the environment cannot be described by a single horizontal cross-section.

3.4.1 RTABMap (LiDAR)

RTABMap [30] (Real-Time Appearance-Based Mapping) is a graph-based SLAM system developed at Université de Sherbrooke. Pose estimates come from ICP-based scan matching against a local point cloud map. The pose graph is globally optimised whenever a loop closure is detected.

Loop closure detection is appearance-based: compact descriptors of each scan are compared against all stored nodes, enabling detection of revisits after long detours (Section 2.7).

To keep computation bounded, memory is split into Working Memory (WM) and Long-Term Memory (LTM). When processing time exceeds a set threshold, the least-visited nodes are moved to LTM. They are only retrieved when a loop closure candidate matches them.

3.5 Tightly-Coupled LiDAR-IMU

These algorithms work with full 3D point clouds with an IMU fused directly inside the same estimator.

The LiDAR provides geometric accuracy to estimate and remove the IMU's accumulated drift. Neither sensor alone is sufficient.

One system is evaluated: FAST-LIO2, an odometry-only system with no loop closure.

A rotating LiDAR produces points over a full revolution while the robot is moving, so points captured at the start and end of the scan come from slightly different poses. Deskewing corrects this by transforming all points in a scan to a common timestamp using IMU-interpolated poses. IMU preintegration [8] avoids re-integrating all high-frequency IMU measurements from scratch at every step by accumulating them into a single compact motion estimate that remains valid during optimisation. FAST-LIO2 relies on both techniques.

3.5.1 FAST-LIO2

FAST-LIO2 [31] was developed at the HKU-MARS laboratory. It registers raw LiDAR points directly to the map. No feature extraction step is needed. This makes it work well with LiDARs that have a small field of view or non-repetitive scanning patterns, where feature-based methods fail.

It uses an iEKF: each point is matched to its nearest neighbours, a local plane is fitted, and the point-to-plane distance drives the update, correcting pose and IMU biases in one step.

The map is stored in an incremental k-d tree (ikd-Tree), which extends the standard k-d tree (Section 2.4.1) to support inserting and deleting points without rebuilding the structure. This keeps search fast as the map grows.

3.6 Visual SLAM

Visual SLAM estimates the trajectory from image sequences using feature-based stereo vision. The fundamental weakness is texture dependency: flat or uniformly coloured surfaces produce few keypoints and the system loses tracking.

Two systems are evaluated, both using a stereo camera pair.

3.6.1 cuVSLAM

cuVSLAM [32] is NVIDIA’s visual SLAM library, distributed as part of the Isaac ROS ecosystem. It runs the entire pipeline on an NVIDIA GPU, which allows dense feature detection and tracking at rates that would be too slow on CPU. Stereo matching produces depth for each tracked feature via triangulation. IMU measurements are fused with the visual estimates to improve tracking stability and robustness to motion blur. Pose increments are estimated from the tracked 3D

feature positions and IMU data, and a pose graph back-end with loop closure maintains global consistency. Because the library is closed-source, internal parameters beyond those exposed through the ROS2 interface are not accessible. The system requires a compatible NVIDIA GPU to run.

3.6.2 RTABMap RGB-D

RTABMap [30] in RGB-D mode reuses the same graph SLAM back-end as the LiDAR variant but uses a visual front-end for loop closure. In this evaluation the depth image comes from stereo matching on the Hawk camera. Wheel odometry provides the motion prior between frames. Loop closure candidates are identified both by spatial proximity in the pose graph and by bag-of-words matching of visual features extracted from the RGB image. Accepted loop closures trigger a global graph optimisation that corrects the accumulated drift. The accumulated map is a dense 3D point cloud built by projecting depth images into the world frame at each pose.

Bibliography

- [1] John J. Leonard and Hugh F. Durrant-Whyte. Mobile robot localization by tracking geometric beacons. *IEEE Transactions on Robotics and Automation*, 7(3):376–382, 1991. [doi:10.1109/70.88147](#). ↑
- [2] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016. [doi:10.1109/TR0.2016.2624754](#). ↑↑↑↑
- [3] Randall Smith, Matthew Self, and Peter Cheeseman. Estimating uncertain spatial relationships in robotics. In Ingemar J. Cox and Gordon T. Wilfong, editors, *Autonomous Robot Vehicles*, pages 167–193. Springer, New York, 1990. [doi:10.1007/978-1-4613-8997-2_14](#). ↑
- [4] Hugh F. Durrant-Whyte. Uncertain geometry in robotics. *IEEE Journal of Robotics and Automation*, 4(1):23–31, 1988. [doi:10.1109/56.767](#). ↑
- [5] Hugh Durrant-Whyte and Tim Bailey. Simultaneous localisation and mapping (SLAM): Part I the essential algorithms. *IEEE Robotics & Automation Magazine*, 13(2):99–110, 2006. [doi:10.1109/MRA.2006.1638022](#). ↑
- [6] Roland Siegwart and Illah R. Nourbakhsh. *Introduction to Autonomous Mobile Robots*. The MIT Press, Cambridge, Massachusetts, 2004. ↑↑↑
- [7] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, 2005. ↑↑↑↑
- [8] Christian Forster, Luca Carlone, Frank Dellaert, and Davide Scaramuzza. On-manifold preintegration for real-time visual–inertial odometry. *IEEE Transactions on Robotics*, 33(1):1–21, 2017. [doi:10.1109/TR0.2016.2597321](#). ↑↑

- [9] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975. [doi:10.1145/361002.361007](#). ↑
- [10] Paul J. Besl and Neil D. McKay. A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992. [doi:10.1109/34.121791](#). ↑
- [11] Yang Chen and Gérard Medioni. Object modelling by registration of multiple range images. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2724–2729, 1991. [doi:10.1109/ROBOT.1991.132043](#). ↑
- [12] Aleksandr Segal, Dirk Hähnel, and Sebastian Thrun. Generalized-ICP. In *Robotics: Science and Systems*, 2009. [doi:10.15607/RSS.2009.V.021](#). ↑
- [13] Peter Biber and Wolfgang Straßer. The normal distributions transform: A new approach to laser scan matching. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2743–2748, 2003. [doi:10.1109/IROS.2003.1249285](#). ↑
- [14] Bradley M. Bell and Frederick W. Cathey. The iterated Kalman filter update as a Gauss–Newton method. *IEEE Transactions on Automatic Control*, 38(2):294–297, 1993. [doi:10.1109/9.250476](#). ↑
- [15] Arnaud Doucet, Nando de Freitas, Kevin Murphy, and Stuart Russell. Rao-Blackwellised particle filtering for dynamic Bayesian networks. In *Proceedings of the Sixteenth Conference on Uncertainty in Artificial Intelligence*, pages 176–183, 2000. ↑
- [16] Giorgio Grisetti, Cyrill Stachniss, and Wolfram Burgard. Improved techniques for grid mapping with Rao-Blackwellized particle filters. *IEEE Transactions on Robotics*, 23(1):34–46, 2007. [doi:10.1109/TR0.2006.889486](#). ↑↑
- [17] Giorgio Grisetti, Rainer Kümmerle, Cyrill Stachniss, and Wolfram Burgard. A tutorial on graph-based SLAM. *IEEE Intelligent Transportation Systems Magazine*, 2(4):31–43, 2010. [doi:10.1109/MITS.2010.939925](#). ↑
- [18] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2d LiDAR SLAM. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, 2016. [doi:10.1109/ICRA.2016.7487258](#). ↑↑

- [19] Dorian Gálvez-López and Juan D. Tardós. Bags of binary words for fast place recognition in image sequences. *IEEE Transactions on Robotics*, 28(5):1188–1197, 2012. [doi:10.1109/TR0.2012.2197158](https://doi.org/10.1109/TR0.2012.2197158). ↑
- [20] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2003. ↑
- [21] Jianbo Shi and Carlo Tomasi. Good features to track. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 593–600, 1994. [doi:10.1109/CVPR.1994.323794](https://doi.org/10.1109/CVPR.1994.323794). ↑
- [22] Davide Scaramuzza and Friedrich Fraundorfer. Visual odometry [Tutorial]. *IEEE Robotics and Automation Magazine*, 18(4):80–92, 2011. [doi:10.1109/MRA.2011.943233](https://doi.org/10.1109/MRA.2011.943233). ↑
- [23] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4):376–380, 1991. [doi:10.1109/34.88573](https://doi.org/10.1109/34.88573). ↑
- [24] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 573–580, 2012. [doi:10.1109/IROS.2012.6385773](https://doi.org/10.1109/IROS.2012.6385773). ↑
- [25] Tong Wu, Liang Pan, Junzhe Zhang, Tai Wang, Ziwei Liu, and Dahua Lin. Density-aware Chamfer distance as a comprehensive metric for point cloud completion. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. [doi:10.48550/arXiv.2111.12702](https://doi.org/10.48550/arXiv.2111.12702). ↑
- [26] Arno Knapitsch, Jaesik Park, Qian-Yi Zhou, and Vladlen Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4):78:1–78:13, 2017. [doi:10.1145/3072959.3073599](https://doi.org/10.1145/3072959.3073599). ↑
- [27] Ignacio Vizzo, Tiziano Guadagnino, Benedikt Mersch, Louis Wiesmann, Jens Behley, and Cyrill Stachniss. KISS-ICP: In defense of point-to-point ICP — simple, accurate, and robust registration if done the right way. *IEEE Robotics and Automation Letters*, 8(2):1029–1036, 2023. [doi:10.1109/LRA.2023.3236571](https://doi.org/10.1109/LRA.2023.3236571). ↑
- [28] MOLA Project Contributors. MOLA: Modular optimization framework for localization and mapping — documentation, 2024. Online; accessed April 2026. <https://docs.mola-slam.org/latest/>. ↑

- [29] Steve Macenski and Ivona Jambrecic. SLAM toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021. [doi:10.21105/joss.02783](https://doi.org/10.21105/joss.02783). ↑
- [30] Mathieu Labbé and François Michaud. RTAB-Map as an open-source lidar and visual SLAM library for large-scale and long-term online operation. *Journal of Field Robotics*, 36(2):416–446, 2019. [doi:10.1002/rob.21831](https://doi.org/10.1002/rob.21831). ↑↑
- [31] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. FAST-LIO2: Fast direct LiDAR-inertial odometry. *IEEE Transactions on Robotics*, 38(4):2053–2073, 2022. Preprint: <https://arxiv.org/pdf/2107.06829>. [doi:10.1109/TR0.2022.3141878](https://doi.org/10.1109/TR0.2022.3141878). ↑
- [32] NVIDIA Isaac ROS Contributors. Isaac ROS Visual SLAM — documentation, 2024. Online; accessed May 2026. https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_visual_slam/index.html. ↑

Appendix A

Appendices